

EEG Seizure Prediction

Generalisation Study — Code Analysis Report

June 2026

Prepared from: Untitled0.ipynb

Abstract: This report documents a Python Jupyter notebook that implements a comprehensive machine learning study on EEG-based seizure prediction. The notebook evaluates three EEG datasets, two preprocessing pipelines, three regularisation strategies, and two class-imbalance handling methods using Logistic Regression as the base classifier. Key findings on generalisation, sparsity, and clinical tradeoffs are summarised.

1. Project Overview

The notebook, titled *EEG Seizure Prediction — Generalisation Study*, is structured into eight sections: installations, dataset collection, preprocessing pipelines, baseline modelling, overfitting/underfitting analysis, regularisation study, imbalance handling, and a comparative summary. It is designed as a reproducible research template that degrades gracefully to high-fidelity simulated data when real datasets are unavailable.

Key Technologies

- **Core ML:** scikit-learn — LogisticRegression, PCA, FastICA, SelectKBest, StratifiedKFold
- **Imbalance:** imbalanced-learn — SMOTE (Synthetic Minority Over-sampling)
- **Signal Processing:** scipy.ndimage.gaussian_filter1d for temporal noise removal
- **Visualisation:** matplotlib and seaborn for publication-quality figures
- **Data:** NumPy / Pandas; optional Kaggle API for real dataset download

Research Questions Addressed

- Q1: Does preprocessing order (Pipeline A vs B) affect generalisation?
- Q2: Which regularisation type generalises best across datasets?
- Q3: Does ElasticNet consistently outperform L1 / L2?
- Q4: How does imbalance handling interact with regularisation?

2. Datasets

Three EEG datasets are used, spanning different seizure prevalence rates and noise profiles. When Kaggle credentials are unavailable, statistically faithful simulations are substituted automatically, preserving class imbalance ratios and feature dimensionality.

Dataset	Samples	Features	Seizure Rate	Source
CHB-MIT Scalp EEG	100,000	178	~5%	Kaggle / PhysioNet
UCI Epileptic Seizure	11,500	178	~21.7%	Kaggle
TUH EEG Seizure Corpus	50,000	178	~1%	Simulated only

Table 1: Dataset characteristics

The CHB-MIT dataset mimics severe class imbalance typical of long-term EEG monitoring. The UCI dataset provides a more balanced benchmark. TUH, with only 1% seizure prevalence, is the most clinically realistic and serves as the primary stress test for imbalance handling.

3. Preprocessing Pipelines

Pipeline A — Reduction-First

Pipeline A prioritises noise removal before feature selection, making it robust to signal artifacts common in scalp EEG.

- **Step 1 — StandardScaler:** Zero-mean, unit-variance normalisation across all 178 features.
- **Step 2 — Gaussian Filter ($\sigma=1.5$):** Per-sample smoothing along the time-series axis to suppress high-frequency noise.
- **Step 3 — SelectKBest ($k=50$):** ANOVA F-test selects the 50 most discriminative features, reducing dimensionality from 178 to 50.

Pipeline B — Extraction-First

Pipeline B applies unsupervised decomposition techniques to learn latent representations, combining source separation principles from neuroscience.

- **Step 1 — MinMaxScaler:** Scales all features to $[0, 1]$.
- **Step 2 — PCA (50 components):** Principal Component Analysis compresses features to 50 orthogonal dimensions.
- **Step 3 — FastICA (20 components):** Independent Component Analysis extracts 20 source-like signals, mimicking BSS approaches common in EEG research.

Key Design Choice: Pipeline A uses supervised feature selection (ANOVA F-test uses labels), while Pipeline B is fully unsupervised. This makes Pipeline A more data-efficient in low-seizure-prevalence settings.

4. Baseline Model: Logistic Regression

A standard L2-penalised Logistic Regression ($C=1.0$, $\text{max_iter}=1000$) is used as the baseline across all dataset/pipeline combinations. Three metrics are reported:

- **Accuracy:** Overall fraction of correctly classified samples.
- **Macro F1 Score:** Unweighted mean F1 across both classes — critical for imbalanced data.
- **PR-AUC (Average Precision):** Area under the precision-recall curve, the primary metric for rare-event detection.

The `evaluate_model()` helper function encapsulates fit, predict, and score computation in a single reusable call, ensuring consistent evaluation across all experiments.

5. Overfitting & Underfitting Analysis

Section 4 of the notebook demonstrates the bias-variance tradeoff using three engineered scenarios on the UCI dataset with Pipeline A features.

Scenario	Configuration	Expected Behaviour
Underfitting	C=0.001, only 5 features	High bias; low train & test F1
Optimal Fit	C=1.0, full pipeline	Balanced; train $\approx$ test F1
Overfitting	C=1e6, polynomial + 50 noise features	Low bias, high variance; large train-test gap

Table 2: Bias-variance scenarios

Learning curves are plotted for each scenario using StratifiedKFold cross-validation (k=3). The curves show training F1 and validation F1 as a function of training set size, making the convergence or divergence between train and validation performance visually apparent.

6. Regularisation Study

Three regularisation strategies are compared across all 6 dataset/pipeline combinations using C=0.1 (moderate regularisation strength):

Regularisation	Solver	Key Property	Expected Use Case
L1 (Lasso)	saga	Sparsity — zeros out irrelevant features	High-dimensional noisy EEG
L2 (Ridge)	lbfgs	Smoothness — shrinks all coefficients equally	Correlated feature groups
ElasticNet	saga	Hybrid L1+L2 (l1_ratio=0.5)	Redundant + sparse mixed settings

Table 3: Regularisation configurations

Stability Analysis

Each configuration is evaluated across 5 random seeds (0, 7, 13, 21, 37). Mean and standard deviation of macro-F1 are reported. Stability heatmaps are produced for both pipelines, enabling identification of configurations sensitive to initialisation.

Sparsity Analysis

The number of non-zero coefficients ($|\text{coef}| > 1\text{e-}6$) is tracked for each model. L1 is expected to produce the sparsest solutions, potentially zeroing entire EEG channels, while L2 retains all features with small but non-zero weights.

7. Class Imbalance Handling

The TUH dataset (1% seizure rate) is used as the stress test. Two complementary strategies are evaluated against a no-handling baseline:

Technique	Mechanism	Regularisation	Clinical Consideration
Baseline (None)	No correction	L2 (C=0.1)	Likely predicts all non-seizure
SMOTE + L2	Generates synthetic seizure samples to balance classes	L2	Higher recall; more false alarms
Class Weight + L1	Amplifies minority-class loss during training	L1	Fewer alarms; safer for deployment

Table 4: Class imbalance handling techniques

The `k_neighbors` parameter for SMOTE is dynamically capped at `min(5, minority_count - 1)` to prevent runtime errors when the minority class is very small, which is particularly important for the TUH dataset at training time.

Evaluation Framework

- Precision-Recall curves with PR-AUC for all three approaches
- Precision at Recall = 0.80 (clinically meaningful operating point)
- Confusion matrices to visualise false positive / false negative tradeoffs

8. Key Findings

Q1: Preprocessing Order

Pipeline A (denoise-first: StandardScaler → Gaussian → SelectKBest) outperforms Pipeline B in most EEG settings. Denoising before feature selection prevents artificially inflated F-scores on noisy features. Pipeline B's ICA step can destroy discriminative signal when the seizure pattern has low variance across independent components.

Q2: Best Regularisation

ElasticNet achieves the highest average F1 across datasets, combining L1 sparsity (removing irrelevant channels) with L2 stability (handling correlated channels). Tuning l1_ratio is critical for optimal performance.

Q3: ElasticNet Consistency

ElasticNet does not always win. On the high-noise TUH dataset (1% seizure), L2 alone can outperform ElasticNet because smooth shrinkage avoids over-sparsification of the rare ictal pattern. After Pipeline B's ICA decorrelation, L1 can also beat ElasticNet.

Q4: Imbalance Tradeoff

SMOTE + L2 increases seizure recall but inflates false positives. Class Weighting + L1 is more conservative, producing fewer false alarms. For clinical deployment (alarm fatigue), Class Weighting + L1 is preferred. For research (maximise detection), SMOTE + ElasticNet is recommended.

9. Code Architecture

The notebook follows a clean functional design pattern. All major operations are encapsulated in reusable functions:

Function / Class	Section	Purpose
install(pkg)	0	Safe pip installer with error suppression
_setup_kaggle()	1	Detects Kaggle credentials; returns bool
load_chbmit()	1	Loads or simulates CHB-MIT (100k samples)
load_uci_epilepsy()	1	Loads or simulates UCI (11.5k samples)
load_tuh_eeg()	1	Always simulates TUH (50k samples, 1%)

<code>_gaussian_filter_rows()</code>	2	Per-sample Gaussian smoothing (sigma=1.5)
<code>pipeline_A()</code>	2	StandardScaler → Gaussian → SelectKBest
<code>pipeline_B()</code>	2	MinMaxScaler → PCA(50) → ICA(20)
<code>evaluate_model()</code>	3	Fit + score → (acc, F1, PR-AUC, model)

Table 5: Code architecture — key functions

Visualisations Produced

- **class_distributions.png** — Bar charts of class counts for all three datasets
- **preprocessing_visualization.png** — 2D scatter plots after each pipeline (6 panels)
- **learning_curves.png** — Train vs validation F1 for under/optimal/over fitting
- **sparsity_comparison.png** — Non-zero coefficient counts by regularisation type
- **stability_heatmap.png** — Mean $\pm$ Std F1 across 5 seeds (pipelines A and B)
- **pr_curves_imbalance.png** — Precision-recall curves for TUH imbalance handling
- **confusion_matrices.png** — Confusion matrices for SMOTE vs Class Weighting
- **summary_heatmap.png** — F1 and PR-AUC across all dataset/regularisation configurations

10. Reproducibility & Data Access

The notebook is fully self-contained and reproducible without internet access. `np.random.seed(42)` is set globally, and individual `RandomState` instances are used per dataset to ensure deterministic simulation. All three datasets have clear fallback simulation paths triggered automatically.

Real Data Instructions

- **UCI (Kaggle):** Place `kaggle.json` in `~/kaggle/` and run normally.
- **CHB-MIT (PhysioNet):** Register at physionet.org, download `.edf` files, read with MNE-Python.
- **TUH EEG:** Requires institutional data-use agreement from Temple University Hospital.

11. Conclusion

This notebook provides a rigorous, reproducible framework for EEG seizure prediction research. It covers the full ML pipeline from data loading through preprocessing, regularisation, and imbalance handling, with thorough comparative evaluation. The design choices — statistically faithful simulation, stratified splits, multi-seed stability testing, and PR-AUC as primary metric — reflect best practices for clinical ML with rare events.

Recommended Next Steps: (1) Replace simulated data with real datasets for final results. (2) Extend beyond Logistic Regression to tree-based models (XGBoost, Random Forest) and deep learning. (3) Implement patient-independent cross-validation to test true cross-subject generalisation. (4) Add temporal context with sliding windows for online seizure prediction.